

PP_zsmf_t2

Disclaimer

Zur Erstellung dieses Dokuments wurde sehr viel mit LLMs gearbeitet. Die Informationen können fehlerhaft und unvollständig sein, deshalb unbedingt schon mit einer Grundahnung an die Sache herangehen und nicht ausschließlich die Zusammenfassung zum Lernen, sondern viel eher zum schnellen Wiederholen verwenden!

Inhalt

- [vo6](#)
- [vo7](#)
- [vo8](#)
- [vo9](#)
- [vo10](#)

vo6

Generizität

- **Was ist Generizität?**
 - Eine Form der **Parametrisierung**, bei der "Lücken" im Code (generische Parameter, meist **Typparameter**) erst bei der Verwendung durch konkrete Typen ersetzt werden.
 - Es ist ein **statischer Mechanismus**; die Typinformation wird zur Übersetzungszeit genutzt. Es findet kein dynamisches Binden wie bei Untertypen statt.
- **Wozu Generizität?**
 - **Code-Wiederverwendung**: Anstatt denselben Code für verschiedene Typen (z.B. eine `List` für `String` und eine `List` für `Integer`) mehrfach zu schreiben (Copy-Paste), schreibt man ihn nur einmal generisch (z.B. `List<T>`).
 - **Typsicherheit**: Der Compiler garantiert statisch, dass z.B. in eine `List<String>` nur `String`-Objekte eingefügt werden können. Ohne Generizität (z.B. `List` mit `Object`) würden Typfehler erst zur Laufzeit durch `ClassCastException` bemerkt.

Einfache Generizität in Java

- Generische Klassen oder Interfaces deklarieren Typparameter in spitzen Klammern (z.B. `<A>`).
- Innerhalb der Klasse/des Interface kann der Typparameter (z.B. `A`) wie ein normaler Referenztyp verwendet werden.
- **Wichtig**: Typparameter können in Java **nur durch Referenztypen** (Klassen, Interfaces) ersetzt werden, **nicht durch primitive Typen** (wie `int`, `boolean`).

Beispiel: Interfaces `Collection<A>` und `Iterator<A>`

```
public interface Collection<A> {
    void add(A elem);
    Iterator<A> iterator();
}

public interface Iterator<A> {
    A next();
    boolean hasNext();
}
```

Verwendungsbeispiele:

```
Collection<String>
Collection<Integer>
```

Typ eingesetzt für Typparameter

```
(enthält void add(String elem))
(enthält void add(Integer elem))
```

- `Collection<A>` definiert Methoden, die den Typparameter `A` verwenden (z.B. `void add(A elem)`).
- Bei Verwendung als `Collection<String>` ersetzt der Compiler `A` konzeptuell durch `String`.

Beispiel: Implementierung `List<A>`

```

public class List<A> implements Collection<A> {
    private class Node {
        private A elem; private Node next = null;
        private Node(A elem) { this.elem = elem; }
    }
    private Node head = null, tail = null;
    public void add(A x) { if (head == null) tail = head = new Node(x);
                          else tail = tail.next = new Node(x); }
    public Iterator<A> iterator() {
        return new Iterator<A>() {
            Node p = head;
            public boolean hasNext () { return p != null; }
            public A next () { if (p == null) return null;
                              A elem = p.elem; p = p.next; return elem; }
        };
    }
}

```

```

public class List<A> implements Collection<A> {
    private Node<A> head = null, tail = null;
    public void add(A x) { tail = head==null ? (head = new Node<>(x)) : tail.add(x); }
    public Iterator<A> iterator() { return new ListIter<>(head); }
}
class Node<A> {
    private final A elem; private Node<A> next = null;
    Node(A elem) { this.elem = elem; }
    A getElem() { return elem; }
    Node<A> getNext() { return next; }
    Node<A> add(A elem) { return next = new Node<>(elem); }
}
class ListIter<A> implements Iterator<A> {
    private Node<A> p;
    ListIter(Node<A> p) { this.p = p; }
    public boolean hasNext() { return p != null; }
    public A next() { if (p == null) return null;
                     A elem = p.getElem(); p = p.getNext(); return elem; }
}

```

- Die Klasse `List<A>` implementiert `Collection<A>`.
- Sie verwendet eine Hilfsklasse `Node<T>`. Der Typparameter `T` in `Node` ist unabhängig von `A` in `List`, wird aber bei der Verwendung (z.B. `private Node<A> head`) mit `A` gleichgesetzt.
- Konstruktoren (z.B. `new Node<A>(elem)`) geben selbst keine Typparameter an, aber bei der Objekterzeugung (`new`) muss der Typ angegeben werden.
- `ListIter` ist eine **innere Klasse** und kann direkt auf den Typparameter `A` der umschließenden `List`-Klasse zugreifen.

Beispiel: Verwendung von `List<A>`

```

class ListTest {
    public static void main(String[] args) {
        List<Integer> xs = new List<Integer>();
        xs.add(new Integer(0));
        Integer x = xs.iterator().next();

        List<String> ys = new List<String>();
        ys.add("zerro");
        String y = ys.iterator().next();

        List<List<Integer>> zs = new List<List<Integer>>();
        zs.add(xs);
        // zs.add(ys); ! Compiler meldet Fehler !
        List<Integer> z = zs.iterator().next();
    }
}

```

- `List<Integer> xs = new List<Integer>();`
- `xs.add(0);`
 - Hier funktioniert `0` (primitiver Typ `int`), weil Java **Autoboxing** durchführt (automatische Umwandlung von `int` zu `Integer`).
- `Integer x = xs.iterator().next();`
 - Das Ergebnis von `next()` ist dank Generizität `Integer`, kein `Object`. Es ist kein Cast nötig.
- `zs.add(ys);` // !! compiler shows error message !!
 - Der Compiler verhindert, dass eine `List<String>` (ys) in eine `List<List<Integer>>` (zs) eingefügt wird. Die Typsicherheit ist statisch garantiert.

Generische Methoden

- Auch einzelne Methoden können Typparameter deklarieren, unabhängig davon, ob die Klasse selbst generisch ist.
- Der Typparameter wird *vor* dem Rückgabetypp deklariert (z.B. `<A>`).

Beispiel: `Comparator<A>` und `max()`


```

public interface Comparator<A> {
    int compare(A x, A y);
}
public class CollectionOps {
    public static <A> A max(Collection<A> xs, Comparator<A> c) {
        Iterator<A> xi = xs.iterator();
        A w = xi.next();
        while (xi.hasNext()) {
            A x = xi.next();
            if (c.compare(w, x) < 0)
                w = x;
        }
        return w;
    }
}

```

- `public interface Comparator<A>` definiert eine Schnittstelle zum Vergleichen von Objekten des Typs `A`.
- `public static <A> A max(Collection<A> xs, Comparator<A> c)`
 - `<A>` deklariert den Typparameter für diese Methode.
 - Die Methode findet das größte Element in einer `Collection<A>` gemäß einem `Comparator<A>`.
- **Typinferenz bei Methoden:**
 - Beim Aufruf `CollectionOps.max(xs, cx)` (wobei `xs` eine `List<Integer>` und `cx` ein `Comparator<Integer>` ist) muss `A` nicht explizit angegeben werden.
 - Der Compiler **leitet (inferiert)** aus den Argumenten (`xs` und `cx`) ab, dass `A` in diesem Aufruf `Integer` sein muss.

Gebundene Generizität (Bounded Generics)

- Oft reicht ein beliebiger Typ nicht aus. Man muss sicherstellen, dass der Typ bestimmte Methoden besitzt.
- **Schranken (Bounds):** Schränken ein, welche Typen einen Typparameter ersetzen dürfen.
- **Syntax:** `<T extends SomeClass & InterfaceA & InterfaceB>`
 - `extends` wird immer verwendet (auch für Interfaces).
 - Nur Typen, die **Untertypen aller Schranken** sind, sind erlaubt.
 - Im Rumpf der generischen Klasse/Methode kann auf alle `public` Methoden/Variablen der Schranken zugegriffen werden.
 - Ist keine Schranke angegeben (ungebunden), wird implizit `Object` als Schranke angenommen.

Beispiel: `Scene<T extends Scalable>`

```

public interface Scalable {
    void scale(double factor);
}

public class Scene<T extends Scalable> implements Iterable<T> {
    public void addSceneElement(T e) { ... }
    public Iterator<T> iterator() { ... }
    public void scaleAll(double factor) {
        for (T e : this)
            e.scale(factor);
    }
    ...
}

```

- `<T extends Scalable>`: Der Typparameter `T` ist auf Typen beschränkt, die `Scalable` implementieren.
- Dadurch ist der Compiler sicher, dass Objekte vom Typ `T` die Methode `scale()` besitzen, und erlaubt den Aufruf `e.scale(factor);`.

Rekursion in Schranken (F-gebundene Generizität):

```

class CollectionOps2 {
    public static <A extends Comparable<A>>
        A max(Collection<A> xs) {
        Iterator<A> xi = xs.iterator();
        A w = xi.next();
        while (xi.hasNext()) {
            A x = xi.next();
            if (w.compareTo(x) < 0)
                w = x;
        }
        return w;
    }
}

```

- `<A extends Comparable<A>>`: Dies ist eine häufige, rekursive Schranke.
- Sie bedeutet: "Erlaube jeden Typ `A`, der mit sich *selbst* vergleichbar ist" (z.B. `Integer implements Comparable<Integer>`).
- Dies ermöglicht der `max`-Methode, die `compareTo`-Methode auf den Elementen selbst aufzurufen (`w.compareTo(x)`).

Keine impliziten Untertypbeziehungen!

- **WICHTIG:** Generizität unterstützt **keine** impliziten Untertypbeziehungen.
- Nur weil `String` ein Untertyp von `Object` ist, ist `List<String>` **KEIN** Untertyp von `List<Object>`.

```
public class Loophole {
    public static String loophole(Integer x) {
        Object[] xs = new String[10];           // no error
        xs[0] = x;                               // exception at runtime
        return xs[0];
    }
}
```

- Arrays verletzen dieses Prinzip (sind *kovariant*): `Object[] ys = xs;` (wobei `xs` ein `String[]` ist) ist erlaubt.
- Dies führt zu Laufzeitfehlern: `ys[0] = y;` (wobei `y` ein `Integer` ist) wirft eine `ArrayStoreException`.

```
public class NoLoophole {
    public static String loophole(Integer x) {
        List<Object> xs = new List<String>();      // error
        xs.add(x);
    }
}
```

- Generics verhindern dies: `List<Object> ys = xs;` (wobei `xs` eine `List<String>` ist) erzeugt einen **Compile-Zeit-Fehler**.

Wildcards (flexible Schranken)

Um die strikte Invarianz (`List<String>` ist kein `List<Object>`) zu lockern, gibt es Wildcards.

- `? extends T` (**Kovarianz, Obergrenze**):
 - "Ein unbekannter Typ, der *maximal* `T` ist (also `T` oder ein Untertyp)."
 - Wird für **Lesezugriffe** (lesende Position, z.B. Rückgabewerte) verwendet.
 - Man kann daraus lesen (man erhält garantiert ein `T`), aber man kann **nichts hineinschreiben** (außer `null`), da der Compiler den genauen Typ nicht kennt (es könnte eine `List<Triangle>` sein, in die man kein `Square` einfügen darf).
 - Beispiel: `void drawAll(List<? extends Polygon> p)` akzeptiert `List<Polygon>`, `List<Triangle>`, `List<Square>`.
- `? super T` (**Kontravarianz, Untergrenze**):
 - "Ein unbekannter Typ, der *mindestens* `T` ist (also `T` oder ein Obertyp)."
 - Wird für **Schreibzugriffe** (schreibende Position, z.B. Eingangsparameter) verwendet.
 - Man kann hineinschreiben (ein `Square` passt immer in eine `List<? super Square>`, egal ob es `List<Square>`, `List<Polygon>` oder `List<Object>` ist), aber man kann **nichts typsicher lesen** (außer als `Object`).
 - Beispiel: `void addSquares(List<? super Square> to)`
- `<?>` (**Ungebundenes Wildcard**):
 - Entspricht `<? extends Object>`. Man kann nur `Object` lesen und (fast) nichts schreiben.

Verwendung von Generizität

Richtlinien (Faustregeln)

- **Gleich strukturierte Klassen/Methoden:** Immer verwenden, wenn dieselbe Logik für verschiedene Typen gebraucht wird, z.B. **Containerklassen** (`List`, `Map`, `Set`).
- **Bibliotheken:** Klassen und Methoden in Bibliotheken sollten (fast) immer generisch sein.
- **Erkennen gleicher Strukturen:** Auch wenn es nicht wie ein Container aussieht: Wenn mehrere Variablen denselben (aber nicht von Anfang an fixen) Typ haben müssen (z.B. Konten für verschiedene, aber pro Konto konsistente Währungen), ist (gebundene) Generizität sinnvoll.
- **Abfangen erwarteter Änderungen:** Wenn erwartet wird, dass sich Parametertypen in Zukunft ändern (spezialisieren), kann Generizität helfen.
- **Generizität vs. Untertypen:** Sie ergänzen sich. Untertypen ermöglichen *heterogene* Listen (verschiedene Typen, ein gemeinsamer Obertyp). Generizität ermöglicht *homogene* Listen (alle Elemente vom exakt gleichen, aber austauschbaren Typ).
- **Laufzeiteffizienz:** Sollte **kein** Entscheidungsgrund für oder gegen Generizität sein (die Unterschiede sind meist minimal oder unerwartet).
- **Natürlichkeit:** Oft ist die "natürlichere" Lösung (basierend auf Erfahrung) die bessere.

Arten der Generizität (Übersetzung)

- **Homogene Übersetzung (Java, C#):**
 - Nennt sich **Type Erasure** (Typauslöschung).
 - Der Compiler übersetzt *eine* generische Klasse (z.B. `List<T>`) in *eine* nicht-generische `.class`-Datei (z.B. `List`).
 - Alle Typparameter (`T`) werden im Bytecode durch ihre Schranke (oder `Object`, falls ungebunden) ersetzt.
 - Der Compiler fügt **automatisch Typumwandlungen (Casts)** an den nötigen Stellen ein (z.B. `(String) list.get(0)`).
- **Heterogene Übersetzung (C++ Templates):**
 - Nennt sich **Code-Duplizierung**.
 - Der Compiler erzeugt für *jede* verwendete Typkombination (z.B. `List<String>`, `List<int>`) **neuen, separaten Maschinencode**.
 - **Vorteil:** Effizienter zur Laufzeit (keine Casts, keine Wrapper-Objekte für primitive Typen wie `int`).
 - **Nachteil:** Größere Programmdateien ("Code Bloat").

Typabfragen und Typumwandlungen

Verwendung dynamischer Typinformation

Zur Laufzeit kann auf Typinformationen zugegriffen werden.

- `obj.getClass()`: Gibt ein `Class`-Objekt zurück, das den **dynamischen Typ** des Objekts repräsentiert. Nützlich für Reflexion.
- `obj instanceof Typ`:

- Prüft, ob der **dynamische Typ** von `obj` ein **Untertyp** von `Typ` ist.
- Liefert `true`, wenn `obj` eine Instanz von `Typ` oder einer seiner Unterklassen ist.
- Liefert `false`, wenn `obj` `null` ist oder kein Untertyp ist.
- Beispiel: `if (p instanceof Student)`
- **Typumwandlung (Cast):** `(Typ)obj`
 - Teilt dem Compiler mit, dass `obj` (auch wenn nur als Obertyp deklariert) als `Typ` behandelt werden soll.
 - **Down-Cast:** (Vom Obertyp zum Untertyp, z.B. `(Student)person`)
 - Ist **riskant**.
 - Wird zur **Laufzeit** überprüft.
 - Schlägt fehl (wirft `ClassCastException`), wenn der dynamische Typ von `obj` kein Untertyp von `Typ` ist.
 - **Up-Cast:** (Vom Untertyp zum Obertyp, z.B. `(Person)student`)
 - Ist **immer sicher** und wird nicht zur Laufzeit geprüft.

Gefahren von Typabfragen und Casts

- **Umgehung der statischen Typsicherheit:** Sie schalten die Garantien des Compilers aus. Fehler (z.B. eine `ClassCastException`) passieren erst zur Laufzeit.
- **Schlechte Wartbarkeit:**
 - Der Code wird "fix verdrahtet" auf spezifische Typen.
 - `if (x instanceof T1) ... else if (x instanceof T2) ...`
 - Wenn ein neuer Typ `T3` hinzukommt, müssen **alle** diese `instanceof`-Ketten im gesamten Programm gefunden und manuell erweitert werden.
 - **Bessere Alternative: Dynamisches Binden** verwenden (eine Methode `doSomething()` definieren und in `T1`, `T2`, `T3` unterschiedlich implementieren; statt `if (x instanceof ...)` einfach `x.doSomething()` aufrufen).

Faustregel: Typabfragen und Typumwandlungen (Casts) sollen nach Möglichkeit **vermieden** werden.

Typumwandlungen und Generizität (Type Erasure)

- **Homogene Übersetzung (Type Erasure):**
 - `List<String>` wird im Bytecode zu `List` (dem "Raw Type").
 - `T` wird zu `Object` (oder der Schranke).
- **Compiler fügt Casts ein:**
 - Ein generischer Aufruf `String y = xs.iterator().next();` (wobei `xs` eine `List<String>` ist)...
 - ...wird vom Compiler übersetzt in: `String y = (String) xs.iterator().next();`
 - Der Compiler garantiert (dank seiner statischen Prüfungen), dass dieser Cast zur Laufzeit **immer** erfolgreich sein wird (solange keine Raw Types verwendet werden).

- **"Sichere" Typumwandlungen:**
 1. **Up-Casts:** (Immer sicher).
 2. Casts nach einer `instanceof`-Prüfung.
 3. Casts, die der Compiler bei Generizität einfügt (sind statisch geprüft).
 - **Gefahr: Raw Types (Typen ohne `<...>`):**
 - Wenn man `List` (Raw Type) statt `List<String>` verwendet, schaltet man die statischen Prüfungen der Generizität aus. Der Compiler fügt keine (oder falsche) Casts ein, was zu `ClassCastException`s führt.
 - **`instanceof` mit Generizität:**
 - `if (o instanceof List<String>) // COMPILE-FEHLER!`
 - **Warum?** Wegen Type Erasure. Zur Laufzeit gibt es keine Information über `<String>`. Das System kann eine `List<String>` nicht von einer `List<Integer>` unterscheiden.
 - **Erlaubt ist nur:** `if (o instanceof List)` (Prüfung auf den Raw Type).
-

Kovariante Probleme

- **Problem:** Situationen, in denen man sich (aus "natürlicher" Sicht) **kovariante Eingangsparameter** wünscht, obwohl die Theorie der Ersetzbarkeit **Kontravarianz** vorschreibt.
- **Beispiel (Tiere füttern):**
 - `class Animal { abstract void eat(Food x); }`
 - `class Cow extends Animal { ... }`
 - `class Grass extends Food { ... }`
 - Man **möchte** `Cow.eat(Grass x)` überschreiben.
 - Das ist aber **kovariant** zu `Animal.eat(Food x)` (da `Grass` ein Untertyp von `Food` ist).
 - Dies ist **nicht erlaubt**, da es die Ersetzbarkeit verletzt (ein `Animal`-Client könnte versuchen, einer `Cow` `Meat` zu geben).
- **Lösung (Simulation mit `instanceof`):**
 - Die Methode in `Cow` behält die invariante Signatur `eat(Food x)`.
 - Innerhalb der Methode wird der dynamische Typ des Arguments geprüft.
 - Dies kann mit **Überladen** kombiniert werden, um Aufrufern, die den spezifischen Typ (`Grass`) kennen, einen direkten (effizienteren) Einstiegspunkt zu geben, während der allgemeine (polymorphe) Aufruf über `eat(Food x)` weiterhin funktioniert.

vo7

Kovariante Probleme

Das Grundproblem bei kovarianten Parametern ist, dass die Spezialisierung von Parametertypen in Unterklassen in vielen statisch typisierten Sprachen (wie Java) zu Problemen mit der Typsicherheit führt.

Ausgangssituation: Einfache Typhierarchie

Es gibt eine Hierarchie von Nahrung (`Food`) und Tieren (`Animal`).

- `Grass` und `Meat` sind Untertypen von `Food`.
- `Cow` und `Tiger` sind Untertypen von `Animal`.

```
abstract class Food { ... }

class Grass extends Food { ... }

class Meat extends Food { ... }
```

Das Problem

Ein `Animal` hat eine Methode `eat(Food x)`.

- Eine `Cow` frisst eigentlich nur `Grass`.
- Ein `Tiger` frisst eigentlich nur `Meat`.
- Man möchte in der Unterklasse den Parametertyp einschränken (kovariant anpassen), z.B. `void eat(Grass x)` in `Cow`.

Problem in Java: Java erlaubt keine kovarianten Parametertypen beim Überschreiben (Overriding). Die Methode muss exakt `eat(Food x)` heißen, um die Methode der Oberklasse zu überschreiben.

Lösungsversuch 1: Dynamische Typabfrage

Man behält die Signatur `eat(Food x)` bei und prüft zur Laufzeit den Typ.

⚠ Nachteile

- Fehler treten erst zur Laufzeit auf (Laufzeitfehler statt Compilezeitfehler).
- Code wird durch `instanceof` und Casts unübersichtlich.
- Verletzung des Prinzips, dass Unterklassen Objekte der Oberklasse vollständig ersetzen können sollten (Liskovsches Substitutionsprinzip), da `Cow` bei falschem Futter "erkrankt" (`fallIll()`).

```

abstract class Animal {
    public abstract void eat(Food x);
    ...
}
class Cow extends Animal {
    public void eat(Food x) {
        if (x instanceof Grass) { ... } else fallIll();
    }
}
class Tiger extends Animal {
    public void eat(Food x) {
        if (x instanceof Meat) { ... } else showTeeth();
    }
}

```

Überladen (Overloading)

Ein weiterer Versuch ist das Überladen der Methode. Man definiert in `Cow` zusätzlich `eat(Grass x)`.

```

class Grass extends Food { ... }

abstract class Animal {
    public abstract void eat(Food x);
}

class Cow extends Animal {
    public void eat(Grass x) { ... }
    public void eat(Food x) {
        if (x instanceof Grass) eat((Grass)x);
        else fallIll();
    }
}

```

Statisches Binden bei Überladen

Überladen funktioniert über **statisches Binden**. Das bedeutet, der Compiler entscheidet anhand des **deklarierten Typs** der Variablen, welche Methode aufgerufen wird, nicht anhand des dynamischen Typs (des echten Objekts im Speicher).

Überladen = statisches Binden

Nur die deklarierten Typen sind entscheidend.

Beispiel:

```
Cow cow = new Cow();
Food grass = new Grass(); // Deklarierter Typ ist Food!
cow.eat(grass);
```

- Hier wird `eat(Food x)` aufgerufen, da `grass` als `Food` deklariert ist.
- Es wird **nicht** die spezialisierte Methode `eat(Grass x)` aufgerufen, obwohl das Objekt eigentlich `Grass` ist.
- Um `eat(Grass x)` aufzurufen, müsste man explizit casten: `cow.eat((Grass)grass)`.

nur deklarierte Typen für Überladen entscheidend:

```
Cow cow = new Cow();
Food grass = new Grass();
cow.eat(grass);           // eat(Food x) in Cow
cow.eat((Grass)grass);    // eat(Grass x) in Cow
```

Achtung: deklarierten Typ von cow betrachten:

```
Animal cow = new Cow();
Food grass = new Grass();
cow.eat(grass);           // eat(Food x) in Cow
cow.eat((Grass)grass);    // eat(Food x) in Cow !!
```

🔗 Empfehlung (aus dem Skriptum/Folien)

- Überladen in Typhierarchien vermeiden (besonders wenn Methoden der Oberklasse überladen werden).
- Es ist fehleranfällig, da der Unterschied zwischen statischem und dynamischem Binden oft übersehen wird.
- Unterscheidung von Methoden sollte idealerweise nicht von Typen abhängen, die in einer Vererbungsbeziehung stehen.

Multimethoden

Multimethoden (Multiple Dispatch) wären die ideale Lösung, werden aber von Java **nicht** unterstützt.

- **Prinzip:** Die Methodenauswahl erfolgt zur Laufzeit basierend auf den dynamischen Typen **aller** Argumente (und des Empfängerobjekts).
- Das würde bedeuten, dass `cow.eat(grass)` automatisch die passendste Methode `eat(Grass)` wählt, auch wenn `grass` als `Food` deklariert ist.

```
class Cow extends Animal {
    public void eat(Grass x) { ... }
    public void eat(Food x) {
        fallIll();
    }
}
```

Visitor Pattern (Double Dispatch)

Da Java nur Single Dispatch (Auswahl der Methode nur anhand des Empfängerobjekts `this`) unterstützt, simuliert man Multimethoden durch **Double Dispatch**. Dies ist der Kern des **Visitor Patterns**.

Funktionsweise

Man dreht den Aufruf um, um den dynamischen Typ des Parameters "aufzulösen".

1. **Erster Dispatch (in `Animal`):** Man ruft `eat(Food f)` auf.
2. **Zweiter Dispatch (in `Food`):** Das `Animal`-Objekt übergibt sich selbst (`this`) an das `Food`-Objekt. Das `Food`-Objekt ruft dann eine spezifische Methode für das Tier auf (z.B. `eatenByCow`).

Simulation in Element-Klassen (`Animal`):

```
abstract class Animal {
    public abstract void eat(Food food);
    ...
}
class Cow extends Animal {
    public void eat(Food food) {
        food.eatenByCow(this);
    }
}
class Tiger extends Animal {
    public void eat(Food food) {
        food.eatenByTiger(this);
    }
}
```

Simulation in Visitor-Klassen (`Food`):

```
abstract class Food {
    public abstract void eatenByCow(Cow cow);
    public abstract void eatenByTiger(Tiger tiger);
}
class Grass extends Food {
    public void eatenByCow(Cow cow) { ... }
    public void eatenByTiger(Tiger tiger)
        { tiger.showTeeth(); }
}
class Meat extends Food {
    public void eatenByCow (Cow cow)
        { cow.fallIll(); }
    public void eatenByTiger(Tiger tiger) { ... }
}
```

Struktur des Visitor Patterns

- **Visitor** (`Food`): Definiert Schnittstellen für alle konkreten Elemente (`eatenByCow`, `eatenByTiger`).
- **ConcreteVisitor** (`Grass`, `Meat`): Implementiert die Logik für die Kombination (z.B. was passiert, wenn `Cow` `Grass` isst).
- **Element** (`Animal`): Definiert die `accept`-Methode (hier `eat`), die den Visitor empfängt.
- **ConcreteElement** (`Cow`, `Tiger`): Ruft die entsprechende Methode auf dem Visitor auf (`food.eatenByCow(this)`).

Vor- und Nachteile

- **Pro:** Neue Operationen (neue `Food`-Arten/Visitors) sind einfach hinzuzufügen. Man erstellt einfach eine neue Klasse, die das Visitor-Interface implementiert.
- **Contra:** Die Struktur der Elemente (`Animal`-Hierarchie) muss stabil sein. Wenn eine neue Tierart (`Goat`) hinzukommt, muss das Visitor-Interface (`Food`) und **alle** seine Implementierungen (`Grass`, `Meat`) geändert werden (`eatenByGoat` hinzufügen).

Annotationen

Annotationen sind Metadaten im Code, die Informationen für den Compiler oder die Laufzeit bereitstellen, ohne die Semantik des Programms direkt zu ändern (es sei denn, Tools reagieren darauf).

Annotation ist optionaler Parameter, der

an (fast) beliebige Sprachkonzepte anheftbar ist,
im Java-Code statisch gesetzt wird,
von gesamter Werkzeugkette bis zur Laufzeit auslesbar ist.

```
@Override
public String toString() { ... }

@BugFix(who="Kaspar", date="2023-11-20", level=3,
        bug="class unnecessary and maybe harmful",
        fix="contents of class body removed")
public class Buggy { }
```

Wichtige Standard-Annotationen

- `@Override`: Stellt sicher, dass eine Methode tatsächlich eine Methode der Oberklasse überschreibt (Compilerfehler, wenn nicht).
- `@Deprecated`: Markiert veralteten Code (Compilerwarnung bei Verwendung).
- `@SuppressWarnings`: Unterdrückt Warnungen.

Definition eigener Annotationen

Eigene Annotationen werden mit `@interface` definiert.

```
@Retention(RUNTIME)                // @Retention(value=RUNTIME)
@Target(value=ANNOTATION_TYPE)
public @interface Retention {
    RetentionPolicy value();
}

public enum RetentionPolicy { CLASS, RUNTIME, SOURCE }

@Retention(value=RUNTIME)
@Target(value=ANNOTATION_TYPE)
public @interface Target {
    ElementType[] value();
}

public enum ElementType { ANNOTATION_TYPE, CONSTRUCTOR,
    FIELD, LOCAL_VARIABLE, METHOD, PACKAGE, PARAMETER, TYPE
}
```

- Können Elemente enthalten (wirken wie Methoden, sind aber Datenfelder).
- `default` Werte sind möglich.

Meta-Annotationen

Steuern das Verhalten der Annotation selbst:

- `@Target`: Wo darf die Annotation stehen? (z.B. `ElementType.METHOD`, `ElementType.TYPE`).
- `@Retention`: Wie lange bleibt die Annotation erhalten?
 - `SOURCE`: Nur im Quellcode (für Compiler/Tools), im Bytecode weg.
 - `CLASS`: Im Bytecode, aber zur Laufzeit nicht verfügbar (Standard).
 - `RUNTIME`: Im Bytecode und zur Laufzeit per Reflexion abfragbar.

Zugriff zur Laufzeit (Reflexion)

Wenn `@Retention(RetentionPolicy.RUNTIME)` gesetzt ist, kann man Annotationen zur Laufzeit auslesen.

Annotationen über Reflection zur Laufzeit zugreifbar
(falls `@Retention(RUNTIME)`):

```
String s = "";
BugFix a = Buggy.class.getAnnotation(BugFix.class);
if (a != null) { // null if no such Annotation
    s += a.who() + " fixed a level " + a.level() + " bug";
}

Annotation[] as = Buggy.class.getAnnotations(); // alle
Method[] ms = Buggy.class.getMethods();
// verschiedene analoge Methoden auch auf Method, Field
```

Aspektororientierte Programmierung (AOP)

AOP adressiert **Cross-Cutting Concerns** (Querschnittsbelange). Das sind Funktionalitäten, die sich nicht sauber in eine einzelne Klasse kapseln lassen, sondern über viele Module verstreut sind ("Scattering") und mit der Kernlogik vermischt sind ("Tangling").

Beispiele: Logging, Security (Authentifizierung), Transaktionsmanagement, Tracing.

Grundbegriffe (AspectJ)

Terminologie

- **Join Point:** Ein Punkt im Programmablauf (z.B. Methodenaufruf, Zugriff auf Feld, Exception-Handling), an dem man "einhaken" kann.
- **Pointcut:** Eine Menge von Join Points. Eine Regel, die definiert, *wo* der Aspekt angewendet werden soll (z.B. "alle Methoden in Klasse X").
- **Advice:** Der Code, der an den Join Points ausgeführt werden soll (*was* passieren soll).
Arten: `before`, `after`, `around`.
- **Aspect:** Die Modularisierungseinheit, die Pointcuts und Advice kapselt.

- **Weaving:** Der Prozess, der Aspekte und normalen Code verbindet (z.B. zur Compilezeit oder Ladezeit).

Syntax Beispiele (AspectJ)

Pointcut Definition:

Definiert z.B. alle Aufrufe von Methoden in `javax..` die auf `Listener` enden.

```
public pointcut AddListener(EventListener el) :
    call(* javax..*.add*Listener(EventListener)) && args(el);
```

Pointcut-Typen (Beispiele):

```
execution(Methodensignatur)
call(Methodensignatur)
get(Feldsignatur)
set(Feldsignatur)
this(TypOderIdentifier)
target(TypOderIdentifier)
args(TypOderIdentifier, ...)
cflow(Pointcut)
```

- `execution(...)`: Ausführung der Methode selbst.
- `call(...)`: Aufruf der Methode (beim Aufrufer).
- `within(...)`: Innerhalb einer bestimmten Klasse/Paket.

Advice Definition:

Code, der vor (`before`) oder nach (`after`) dem Pointcut läuft.

```
before() : call(* Account.*(..)) { checkUser(); }
```

```
pointcut connectionOperation(Connection connection) :
    call(* Connection.*(..) throws SQLException)
    && target(connection);
before(Connection connection) :
    connectionOperation(connection) {
        System.out.println("Operation auf " + connection);
    }
```

Vollständiger Aspekt (Tracing Beispiel):

Ein Aspekt, der die Aufruftiefe (`callDepth`) mitzählt und bei jedem Trace-Punkt `before` und `after` etwas ausgibt.

```
public aspect JoinPointTraceAspect {
    private int callDepth = -1;
    pointcut tracePoints() : !within(JoinPointTraceAspect);
    before() : tracePoints() {
        callDepth++;
        print("Before", thisJoinPoint);
    }
    after() : tracePoints() {
        callDepth--;
        print("After", thisJoinPoint);
    }
    private void print(String prefix, Object message) {
        for (int i=0, spaces = callDepth*2; i<spaces; i++)
            System.out.print(" ");
        System.out.println(prefix + ": " + message);
    }
}
```

vo8

Charakter der funktionalen Programmierung

Die funktionale Programmierung erfordert oft ein radikales Umdenken im Vergleich zur prozeduralen oder objektorientierten Programmierung.

- **Keine Seiteneffekte:** Keine Kommunikation über gemeinsame (veränderbare) Variablen.
- **Referenzielle Transparenz:** Aliase sind harmlos, da Original und Kopie nicht unterscheidbar sind (Werte ändern sich nicht).
- **"Sauber":** Aufgesammeltes Wissen geht nie verloren.
- **Funktionen höherer Ordnung:** Funktionen können wie Daten behandelt werden (als Parameter übergeben, zurückgegeben werden). Sie können jede Kontrollstruktur ersetzen.
- **Abstraktion:** Bei hohem Abstraktionsgrad eher λ -Abstraktion oder strukturelle Abstraktion.
- **Kontrollfluss:** Ausschließlich Rekursion statt Schleifen.
- **Typisierung:** Heute meist vollständig statisch typisiert (mit Typinferenz).
- **Lazy-Evaluation:** Einfach umzusetzen (Auswertung erst bei Bedarf).

Mindset

Man sollte vermeiden, den Kontrollfluss oder die Typinferenz im Detail nachvollziehen zu wollen. Stattdessen darf man sich auf den Compiler verlassen (wenn Typen konsistent sind, ist der Code oft fehlerfrei). Es kommt nicht darauf an, wie mächtig Sprachkonzepte sind, sondern wie man sie einsetzt.

Lambdas in Java

Lambdas in Java sind stark an anonyme innere Klassen angelehnt, aber syntaktisch kompakter und mit spezifischen Einschränkungen.

Lambda vs. Anonyme Innere Klasse

Ein Lambda wird oft verwendet, um ein Interface mit einer einzigen Methode zu implementieren (z.B. einen `Iterator`).

Beispiel: Anonyme Innere Klasse


```

public class List<A> implements Collection<A> {
    private Node<A> head = ...;
    public Iterator<A> iterator() {
        return new Iterator<A>() {
            private Node<A> p = head;
            public boolean hasNext() { return p != null; }
            public boolean next() { ... }
        }
    }
    // Compiler erzeugt Klasse, z.B. List$1.class
}

```

- **Funktionales Interface:** Muss implizit gegeben sein (Interface mit genau einer abstrakten Methode).
- **Variablenzugriff:** Keine *veränderbaren* Variablen aus der Umgebung zugreifbar (lokale Variablen müssen effektiv `final` sein).
- **Implementierung:** Compiler erzeugt intern eine Klasse (z.B. `List$1.class`).

Funktionale Interfaces

- **Definition:** Jedes Interface mit nur *einer* abstrakten Methode.
- **Annotation:** `@FunctionalInterface` (optional, aber empfohlen zur Überprüfung).
- **Strukturelle Abstraktion:** Der Name des Interfaces und der Methode ist für das Lambda eigentlich bedeutungslos, es zählt nur die Signatur.

Wichtige vordefinierte Interfaces (`java.util.function`):

aus Sicht von Java:

jedes Interface mit nur einer abstrakten Methode ist funktionales Interface

aus Sicht von Programmierer_innen:

funktionales Interface ist mit `@FunctionalInterface` annotiert,

beruht auf **struktureller Abstraktion** (Namen und Kommentare bedeutungslos)

→ **fundamental verschieden von typischen Interfaces in objektorientierter Programmierung**

- `Function<T, R>`: $R \text{ apply}(T \ t)$ – Wandelt T in R um.
- `BiFunction<T, U, R>`: $R \text{ apply}(T \ t, U \ u)$ – Wandelt T und U in R um.
- `Consumer<T>`: $\text{void } \text{accept}(T \ t)$ – Konsumiert T (Seiteneffekt, keine Rückgabe).
- `Supplier<T>`: $T \text{ get}()$ – Liefert ein T (Erzeuger).
- `Predicate<T>`: $\text{boolean } \text{test}(T \ t)$ – Prüft Bedingung für T.

Beispiele und Syntax

Einfache Lambdas:

```

Consumer<String> p = s -> System.out.println(s);
Function<Integer, String> value = i -> "value = " + i;

```

```
BiFunction<String, Boolean, String> opt = (s, b) -> b ? s : "";
```

Lambdas mit Block:

Wenn der Körper komplexer ist, werden geschweifte Klammern benötigt. Ein `return` ist dann explizit nötig.

```
Consumer<String> p = s -> System.out.println(s);
p.accept("Hello world.");
Function<Integer,String> value = i -> "value = " + i;
p.accept(value.apply(8));
BiFunction<String,Boolean,String> opt = (s,b) -> b ? s : "";
p.accept(opt.apply("maybe", true));

value = i -> { String r = "value = "; // value änderbar
              r += i;                // lokale Variable r änderbar
              p.accept(r);           // p aus Umgebung, nicht änderbar
              return r;
            };
value.apply(6);
```

Methodenreferenzen (::):

Dienen als Kurzschreibweise für Lambdas, die nur eine existierende Methode aufrufen.

- `String::compareTo` entspricht $(s, t) \rightarrow s.compareTo(t)$
- `Objects::equals` entspricht $(a, b) \rightarrow Objects.equals(a, b)$
- `String::new` entspricht $sb \rightarrow new String(sb)$

Mächtigkeit

- Untypisierter λ -Kalkül ist Turing-mächtig.
- Java-Lambdas sind statisch typisiert und können *alleine* (ohne Hilfsmittel wie Y-Kombinator oder Java-Rekursion) keine Rekursion darstellen, da die Typen unendlich groß werden müssten.
- Da Java aber Schleifen und Rekursion als Sprachelemente besitzt, ist die Gesamtsprache natürlich Turing-mächtig.

Java-8-Streams

Streams sind aneinandergereihte Iteratoren mit Zusatzfunktionalität, basierend auf dem `Spliterator`.

Basis: Der Spliterator

Ein `Spliterator<T>` erlaubt das Iterieren und das Aufteilen (für Parallelisierung) von Elementen.

Wichtige Methoden:

- `boolean tryAdvance(Consumer<? super T> action)`: Führt Aktion auf dem nächsten Element aus. Gibt `false` zurück, wenn keine Elemente mehr da sind.
- `void forEachRemaining(Consumer<? super T> action)`: Verarbeitet alle verbleibenden Elemente.
- `Splititerator<T> trySplit()`: Teilt die Elemente in zwei Teile auf (für Parallelverarbeitung).

Java-8-Streams sind aneinandergereihte Iteratoren mit Zusatzfunktionalität (vereinfacht):

```
interface Splititerator<T> {
    // execute action on next element; returns false if no more element
    boolean tryAdvance(Consumer<? super T> action);

    // execute action on all remaining elements
    default void forEachRemaining(Consumer<? super T> action)
        { do { } while (tryAdvance(action)); }

    // divide elements in this in two parts of about equal size;
    // return one part as new Splititerator, continue with other part
    Splititerator<T> trySplit();
    ...
}
```

Unterschied Iterator vs. Splititerator

- **Iterator**: Zugriff von *außen* in einer Schleife (`next`, `hasNext`). Zugriff strikt sequenziell.
- **Splititerator**: Schleife kann *innen* liegen (`forEachRemaining`). Elementverarbeitung direkt via Lambda (`tryAdvance`). Erlaubt Aufspaltung (`trySplit`) und damit Parallelität.

Arten von Stream-Operationen

1. Erzeugende Operationen (Source):

- Erstellen den Stream (z.B. `stream()`, `parallelStream()` aus Collections).
- Basieren auf `Splititerator`-Objekten.

2. Modifizierende Operationen (Intermediate):

- Geben wieder einen Stream zurück (z.B. `map`, `filter`, `flatMap`).
- Nehmen Lambdas als Parameter.
- Arbeiten "Lazy": Sie rufen `tryAdvance` des vorherigen Streams nur bei Bedarf auf.

3. Abschließende Operationen (Terminal):

- Geben *keinen* Stream zurück (z.B. `reduce`, `collect`, `forEach`).
- Stoßen die Berechnung an ("Eager"), indem sie wiederholt `tryAdvance` aufrufen.

Funktionsweise (Lazy Evaluation)

Die Faulheit (Laziness) ergibt sich automatisch aus der Verkettung der Spliteratoren. Eine modifizierende Operation holt sich nur dann ein Element vom Vorgänger, wenn sie selbst vom Nachfolger (oder der terminalen Operation) gefragt wird.

Beispiel: Fakultät mit Streams

```
public static long fact(int n) {  
    return LongStream.rangeClosed(2, n).reduce(1, (i,j) -> i*j);  
}
```

Applikative Programmierung

Hierbei wird versucht, Aufgaben auf bestehende, abstrakte Muster (wie Map-Reduce) zurückzuführen, anstatt imperative Schleifen zu schreiben.

Beispiel: Daten umformen (`toMap`)

Ziel: Verschachtelte Maps erzeugen und zählen.

Applikativer Ansatz (Streams):

```
public static Map<String, Map<String, Long>>  
    toMap(Collection<Set<String>> sales) {  
    return sales.stream()  
        .flatMap(set -> set.stream()  
            .flatMap(p -> set.stream()  
                .filter(q -> !p.equals(q))  
                .map(q -> new AbstractMap.SimpleEntry<>(p, q))  
            )  
        )  
        .collect(Collectors.groupingBy(e -> e.getKey(),  
            Collectors.groupingBy(e -> e.getValue(),  
                Collectors.counting())));  
}
```

- Nutzt `flatMap`, `filter`, `map` und `collect`.
- Deklarativ: Beschreibt *was* gemacht wird, nicht *wie*.

Imperativer Ansatz (Vergleich):

```

public static Map<String, Map<String, Long>>
    toMap2(Collection<Set<String>> sales) {
    Map<String, Map<String, Long>> res = new HashMap<>();
    sales.forEach(set ->
        set.forEach(p -> {
            Map<String, Long> map = res.computeIfAbsent(p,
                k -> new HashMap<>());
            set.forEach(q -> {
                if (!p.equals(q))
                    map.compute(q, (k,v) -> v==null ? 1 : v+1);
            });
        })
    );
    return res;
}

```

- Verschachtelte `forEach` oder `for`-Schleifen.
- Zustandsänderungen in `res` und `map`.

Praxis-Tipps

- **Einschränkung als Vorteil:** Das restriktive Schema (z.B. Map-Reduce) zwingt zu sauberer Struktur.
- **Typinferenz:** IDE-Vorschläge sind oft nicht hilfreich, solange der Ausdruck nicht vollständig ist. Wenn die Typen konsistent sind, ist der Code oft korrekt ("Wer einen Hammer hat...").
- **Variablen:** Handle Variablen so, als wären sie `final`.
- **Mischformen vermeiden:** Bleibe entweder ganz funktional oder ganz prozedural.

Funktionale Formen als Kontrollstrukturen

Funktionale Programmierung kann klassische Kontrollstrukturen (If-Else, Schleifen) ersetzen.

Nachbildung von `ifThenElse` (Boolesche Algebra)

Man kann `True` und `False` als Klassen definieren, die eine Methode `ifThenElse(t, f)` unterschiedlich implementieren.

- `True` gibt `t` zurück.
- `False` gibt `f` zurück.

```

interface Bool {
    <A> A ifThenElse(A t, A f);
    default Bool negate() { return ifThenElse(False.VALUE, True.VALUE); }
    default Bool and(Bool b) { return ifThenElse(b, False.VALUE); }
    default Bool or(Bool b) { return ifThenElse(True.VALUE, b); }
}

final class True implements Bool {
    private True() {}
    public static final True VALUE = new True();
    public <A> A ifThenElse(A t, A f) { return t; }
}

final class False implements Bool {
    private False() {}
    public static final False VALUE = new False();
    public <A> A ifThenElse(A t, A f) { return f; }
}

```

Kurzschlussoperatoren (Short-Circuit) & Lazy Evaluation

Problem bei der obigen Methode: Java wertet Argumente sofort aus (Eager). Um `if (cond) A` `else B` effizient nachzubauen (A oder B soll nur ausgeführt werden, wenn nötig), braucht man **Lazy Evaluation**.

Lösung: `Supplier<T>`

Statt Werten übergibt man Funktionen (`Supplier`), die den Wert erst bei Aufruf von `.get()` berechnen.

```

default <T> T getIfThenElse(Supplier<T> t, Supplier<T> f) {
    return ifThenElse(t, f).get();
}

```

Dadurch lassen sich auch Operatoren wie `and` (`&&`) und `or` (`||`) effizient implementieren.

```

import java.util.function.*;

@FunctionalInterface
interface LazyBool extends Supplier<Bool> {
    static final LazyBool TRUE = () -> True.VALUE;
    static final LazyBool FALSE = () -> False.VALUE;
    default <T> Supplier<T> ifThenElse(Supplier<T> t, Supplier<T> f)
        { return () -> get().ifThenElse(t, f).get(); }
    default LazyBool negate()
        { return () -> get().ifThenElse(False.VALUE, True.VALUE); }
    default LazyBool and(LazyBool b)
        { return () -> get().ifThenElse(b, FALSE).get(); }
    default LazyBool or(LazyBool b)
        { return () -> get().ifThenElse(TRUE, b).get(); }
}

```

Ausführungszeitpunkte

1. **So früh wie möglich (eager):** Normalfall in Java.
2. **So spät wie möglich (lazy):** Nötig für Semantik von Kontrollstrukturen (z.B. Endlosrekursion vermeiden bei If-Else).

Simulation von Schleifen

Auch Schleifen lassen sich durch Rekursion und Funktionskomposition (`compose`) funktional darstellen (`loopWhile`).

```
public static <T,V,R> Function<T,R>
    compose(Function<V,R> f, Function<T,V> g) {
    return t -> f.apply(g.apply(t));
}

public static <T> Function<T,T>
    loopWhile(Function<T,Bool> cond, Function<T,T> iter) {
    Function<T,T> doIt = i -> loopWhile(cond, iter).apply(iter.apply(i));
    Function<T,T> done = i -> i;
    return init -> cond.apply(init).ifThenElse(doIt, done).apply(init);
}
```

Funktionale Elemente und Best Practices

Wiederverwendung

Bevor man eine Funktion höherer Ordnung selbst schreibt (z.B. `arrayMap`), sollte man prüfen, ob es diese nicht schon gibt (`Arrays.setAll`).

```
public static <T> void arrayMap(T[] xs, Function<T,T> f) {
    for (int i = 0; i < xs.length; i++)
        xs[i] = f.apply(xs[i]);
}

public static <T> void arrayMap2(T[] xs, Function<T,T> f) {
    Arrays.setAll(xs, i -> f.apply(xs[i]));
}
```

Optional

`Optional<T>` ist ein Container, der einen Wert enthalten kann oder nicht (vermeidet `NullPointerException`). Es bietet funktionale Methoden ähnlich wie Streams.

Beispiel: Datei öffnen mit Optional-Kette

```
public static Optional<FileReader> openFile(String... path) {
    return Stream.of(path)
        .map(String::trim)
        .reduce((s, t) -> s + "/" + t)
        .map(s -> {try{return new FileReader(s);}
                    catch(java.io.IOException e){return null;}});
}
```

Methoden von `Optional<T>` (unvollständig):

```
boolean isPresent()
T orElse(T other)
T orElseGet(Supplier<? extends T> other)
<U> Optional<U> map(Function<? super T, ? extends U> mapper)
```

Wichtige Methoden:

- `map`: Wandelt Inhalt um, falls vorhanden.
- `flatMap`: Wie `map`, verhindert aber verschachtelte Optionals (`Optional<Optional<T>>`).
- `orElse` / `orElseGet`: Default-Werte.

Currying

Currying zerlegt eine Funktion mit mehreren Argumenten in eine Kette von Funktionen mit je einem Argument.

- Statt $f(x, y)$ nutzt man $f(x)(y)$.
- In Java: `Function<A, Function<B, C>>` statt `BiFunction<A, B, C>`.

$$f = (s, t) \rightarrow s + t$$

$$g = s \rightarrow t \rightarrow s + t$$

```
BiFunction<String,String,String> f = (s, t) -> s + t;
Function<String,Function<String,String>> g = s -> t -> s + t;
```

```
String s = f.apply("a", "b");
String t = g.apply("a").apply("b");
```

- keine funktionalen Interfaces für Methoden mit mehr als einem Parameter nötig
- `BiFunction` existiert nur aus Bequemlichkeit, nicht kompatibel zu `Function`
- Syntax von Lambdas verlangt keinen Zusatzaufwand für Currying
- Aufwand für Auswertung mit Currying etwas größer
- Currying erhöht Flexibilität bei Auswertung, diese Flexibilität ist aber selten nötig
- Typen können mit Currying sehr komplex sein, `var`-Deklarationen dafür nicht anwendbar
- komplexe Typen verringern Fehlerwahrscheinlichkeit

- Syntax von Lambdas unterstützt es ohne Zusatzaufwand.
- Erhöht Flexibilität, ist aber selten nötig.
- Typdefinitionen werden sehr komplex (kein `var` möglich), was aber Fehlerwahrscheinlichkeit verringern kann.

vo9

Threads und Synchronisation in Java

Atomare Ausführung

In Java sind Operationen wie $i++$ oft nicht atomar (sie bestehen aus Lesen, Ändern, Schreiben). Wenn mehrere Threads gleichzeitig darauf zugreifen, können Inkonsistenzen entstehen (Race Conditions).

- **Problem:** Gleichzeitiger Zugriff auf Variablen i und j ohne Schutz führt zu falschen Werten.
- **Lösung:** `synchronized` Blöcke oder Methoden stellen sicher, dass immer nur ein Thread gleichzeitig den Codeabschnitt ausführt.

```
public class Counter {
    private int i = 0, j = 0;

    public synchronized void flip() { i++; j++; }

    public void flip1() { synchronized(this) { i++; j++; } }

    public void flip2() {
        synchronized(this) { i++; }
        synchronized(this) { j++; }
    }
}
```

Verwendung von synchronized

🔑 Faustregeln für nebenläufige Programme

- Zugriff auf Objekt- und Klassenvariablen nur innerhalb von `synchronized`.
- `synchronized` Methoden und Blöcke sollen nur **kurz** laufen, um Blockaden zu minimieren.
- **Volatile:** Für den Zugriff auf **einzelne** Variablen ist kein `synchronized` nötig, wenn die Variable als `volatile` deklariert ist.

⚠️ **Aber: Das macht zusammengesetzte Operationen wie $i++$ **nicht** atomar!**

- **Atomic Klassen:** `java.util.concurrent.atomic.AtomicInteger` (oder `AtomicLong`) bieten atomare Operationen ohne explizites `synchronized`.

Funktionsweise von synchronized (Monitore)

Jedes Java-Objekt besitzt einen **Lock** (Sperre).

- **Sperren:**

- `synchronized` setzt den Lock des Objekts auf den aktuellen Thread.
- Bei `synchronized`-Methoden ist das Objekt `this`.
- Ist der Lock schon vergeben (anderer Thread), muss der aktuelle Thread warten.
- Ist der Lock vom *eigenen* Thread schon gehalten, darf er weitermachen (Reentrant Lock / Rekursion unterstützt).
- **Warten und Wecken** (nur innerhalb von `synchronized` erlaubt):
 - `wait()`: Gibt den Lock frei und legt den Thread in die Warteliste des Objekts ("schlafen legen").
 - `notify()`: Weckt *einen* beliebigen Thread aus der Warteliste auf.
 - `notifyAll()`: Weckt *alle* Threads in der Warteliste auf (sicherer, um Deadlocks/vergessene Threads zu vermeiden).

ⓘ Aufgeweckte Threads müssen warten, bis der Lock wieder frei ist, bevor sie weiterlaufen.

```
public class PrinterDriver {
    private boolean online = false;
    public synchronized void print(String s) {
        while (!online) {
            try { wait(); }
            catch (InterruptedException ex) { return; }
        }
        ... // send s to printer
    }
    public synchronized void onOff() {
        online = !online;
        if (online) notifyAll();
    }
    ...
}
```

```
public class Producer implements Runnable {
    private PrinterDriver t;
    public Producer(PrinterDriver t) { this.t = t; }
    public void run() {
        String s = ....
        for (;;) {
            ... // produce new value in s
            t.print(s); // send s to the printer server
        }
    }
}
```

Nebenläufigkeit in der Praxis

Wegen der hohen Komplexität sollten eigene Low-Level-Lösungen (wait/notify) vermieden werden, wenn möglich.

Alternativen für Standardanwendungen (`java.util.concurrent`):

- **Future / FutureTask:** Für Berechnungen im Hintergrund.
 - Ergebnis wird später abgeholt (Getter wartet, bis fertig).
 - Darf nicht von anderen Berechnungen beeinflusst werden.
- **Executor:** Für viele kleine, unabhängige Aufgaben (z.B. ThreadPool).
 - Führt Aufgaben je nach Ressourcen parallel oder nacheinander aus.
- **Java-8-Streams:** Für parallele Map-Reduce-Aufgaben.

Threadsicherheit von Datenstrukturen

Threadsicherheit

Eine Datenstruktur kümmert sich selbst um die Synchronisation.

- **Gefahr:** Wegen Deadlock-Gefahr sollte man threadsichere Strukturen **nicht** innerhalb von eigenen `synchronized`-Blöcken aufrufen.
- **ConcurrentHashMap:** Threadsicher, effizient bei vielen gleichzeitigen Zugriffen (empfohlen).
- **HashMap:** Nicht threadsicher, aber effizient bei wenig Konkurrenz.
- **Collections.synchronizedMap(...):** Wrapper, der eine Map threadsicher macht.

Praktische Vorgehensweise

Die Strategie hängt von der Abhängigkeit der Teilaufgaben ab:

1. **Unabhängig:** Parallele Ströme, `Executor`, `Future`.
2. **Gemeinsame Daten, unabhängige Reihenfolge:** `ConcurrentHashMap`.
3. **Abhängig:** (Schwierig!) Versuchen, Aufgaben so aufzuteilen, dass sie unabhängig werden.
 - In Phasen aufteilen: `Phaser`.
4. **Notlösung:** Sequenzialisierung (keine Nebenläufigkeit) oder Monitore (`synchronized`).

Faustregel

Synchronisation muss so einfach wie möglich gehalten werden.

Synchronisation und Objektorientierung

Objektorientierung (Kapselung) und Synchronisation stehen oft im Konflikt.

- **Effizienz:** Synchronisation verhindert Parallelität.
- **Deadlocks:** Häufig bei zyklischen Abhängigkeiten.

- **Lösungsansatz:** Lineare Anordnung der Objekte (Lock Ordering). Wenn x vor y kommt, darf man in `synchronized(y)` nicht `synchronized(x)` aufrufen.
- **Vererbungsprobleme (Inheritance Anomaly):**
 - Untertypen dürfen Einschränkungen (wie `wait()`) nicht strenger handhaben als Obertypen, sonst funktionieren Clients nicht mehr wie erwartet.
 - Synchronisation lässt sich schwer vererben.

Prozesse und Interprozesskommunikation

Ein Prozess ist eine Ausführungseinheit des Betriebssystems (außerhalb der Java VM).

Shell und Prozesse

Java kann Systemprozesse starten, ähnlich wie eine Shell.

Wichtige Shell-Konzepte (Unix/Linux):

- `&`: Hintergrundausführung.
- `|`: Pipe (Ausgabe von A wird Eingabe von B).
- `>`, `>>`: Umleitung in Datei (überschreiben / anhängen).

```
java Test arg1 arg2
echo $PATH
cat file
cat <file
cat file >file2
cat file >>file2
cat file &>file2
cat file &>>file2
cat <file >file2
cat file | wc
cat <file | wc >file2
cat <file |& wc >file2

cat file &
cat file ; wc <file
cat file ; wc <file &
cat file >file2 && wc <file2
cat file || cat file2
cat *
cat */*/A*[a-zI].java
cat \*
cat 'a * 2'
cat "a * 2"
cat 'cat file'
/usr/bin/cat ~username/file
```

```
export PATH=/home/me/bin:$PATH
echo ${PATH}
```

```
if ... ; then ... ; else ... ; fi
if test $x = $y; then echo $x; fi
if [ $x = $y ]; then echo $x; fi
```

```
for i in *.java; do javac $i; done
for i in *.java; do (javac $i &); done
for i in *.java; do (javac $i &>'basename $i .java'.out &); done
for f in 'cat file'; do javac $f.java; done
```

Wohin fließen die Daten?

- `cmd > file`: Leitet **Stdout** in Datei um (überschreibt).
- `cmd >> file`: Leitet **Stdout** in Datei um (hängt an).
- `cmd < file`: Liest **Stdin** aus Datei (statt Tastatur).
- `cmd 2> file`: Leitet nur **Stderr** (Fehler) um.
- `cmd &> file`: Leitet **beides** (Stdout + Stderr) in Datei um.

Verbindet Ausgabe von A mit Eingabe von B. **Laufen immer gleichzeitig (parallel)!**

- `A | B`: Leitet nur **Stdout** von A an B weiter. Fehler landen am Bildschirm.
- `A |& B`: Leitet **Stdout UND Stderr** von A an B weiter.

Wie und wann laufen Befehle?

- `A ; B`: **Sequenz**. Erst A fertigmachen, dann B (egal ob A geklappt hat).
- `A && B`: **UND**. B läuft nur, wenn A **erfolgreich** war (Exit Code 0).
- `A || B`: **ODER**. B läuft nur, wenn A **fehlgeschlagen** ist (Exit Code != 0).
- `A &`: **Hintergrund**. A startet, Shell wartet nicht, User kann sofort weitertippen.

Dateien und I/O-Ströme in Java

Kommunikation erfolgt oft über Ströme (Streams):

- **Standard-Streams**: `System.in` (Input), `System.out` (Output), `System.err` (Error).
- **Ressourcen-Management**: Streams müssen geschlossen werden (`close()`).
 - Am besten mit `try-with-resources`: `try (FileReader r = ...) { ... }`.
- **Pufferung**:
 - Gepufferte I/O (Buffered) ist effizienter.
 - Ungepufferte I/O ist für direkte Weiterleitung.
 - `flush()`: Erzwingt das Schreiben der Daten aus dem Puffer (wichtig bei Kommunikation, damit Daten nicht "hängenbleiben").

Serialisierung

Umwandlung von Objekten in einen Bytestrom (zum Speichern oder Versenden).

- **Interface Serializable**: Markiert Objekte, die automatisch serialisiert werden können.
- **Methoden**: `writeObject(...)` und `readObject(...)`.
- **transient**: Variablen, die mit `transient` markiert sind, werden bei der Serialisierung ignoriert (nicht gespeichert).

Voraussetzung

Auf beiden Seiten (Sender/Empfänger) müssen die gleichen Klassen (.class) vorhanden sein.

Zugriff auf Betriebssystem-Ressourcen

Die Klasse `Runtime` erlaubt Interaktion mit dem OS.

- `Runtime.getRuntime().exec("befehl")`: Startet einen neuen **Prozess** (keinen Thread!).
- **Process Objekt:**
 - `p.getInputStream()`: Liest die Ausgabe des Prozesses (Standardausgabe).
 - `p.getOutputStream()`: Schreibt in die Eingabe des Prozesses.
 - `p.waitFor()`: Wartet auf Beendigung.
 - `p.destroy()`: Bricht den Prozess ab.

über `System.getenv()` Shell-Variablen zugreifbar (`Map<String,String>`)

`Runtime.getRuntime` ermöglicht über Ergebnis Zugriff auf Java-Laufzeitumgebung

- `availableProcessors()` liefert Anzahl nutzbarer Prozessor-Kerne
- `freeMemory()` liefert Größe des verbliebenen Speichers
- `gc()` empfiehlt Garbage-Collector, mehr Speicher freizugeben
- `exec(...)` führt über Betriebssystem neuen Prozess (nicht Thread) aus
- z.B.: `Process p = Runtime.getRuntime().exec("java -cp /home/me/java Test")`

über `p` vom Typ `Process` sind Verbindungen zum Prozess herstellbar

- `p.getOutputStream()` liefert Strom, der mit Standardeingabe von `p` verbunden (Pipeline)
- `p.getInputStream()` liefert Strom, der mit Standardausgabe von `p` verbunden
- `p.getErrorStream()` liefert Strom, der mit Fehlerausgabe von `p` verbunden
- `p.waitFor()` wartet auf Beendigung von `p`, liefert Return-Status
- `p.destroy()` kann `p` abbrechen

Worauf bei Interprozesskommunikation zu achten ist

- **Blockierung:** Unendliches Warten auf Daten vermeiden.
- **Formate:** Sender und Empfänger müssen gleiches Format und Kodierung nutzen.
- **Puffer:** Auf `flush()` achten, sonst kommen Daten eventuell verzögert an.
- **Debugging:** Sehr aufwendig. Fehlermeldungen am besten in eine Datei umleiten.
- **Komplexität:** Verhalten so einfach wie möglich halten. `exec(...)` ist nicht so mächtig wie eine echte Shell (keine Wildcards wie `*.java` direkt möglich, dafür muss man eine Shell aufrufen: `exec("/bin/sh -c ...")`).

vo10

Software-Entwurfsmuster (Design Patterns)

Grundlagen

Zweck:

- Benennen wiederkehrender Probleme und Lösungen.
- Austausch und Wiederverwendung von Erfahrungen (wo Code-Wiederverwendung versagt).
- Sehr abstrakt, daher häufig einsetzbar.

Bestandteile eines Musters:

- Name
- Problemstellung
- Lösung
- Konsequenzen
- (Implementierungshinweise)

Verwendung

- **Empfohlen:** Konsequenzen für den Entwurf abschätzen; Erfahrungsschatz aufbauen.
- **Nicht empfohlen:** Muster pauschal als "guten Lösungsansatz" missverstehen oder möglichst viele auf einmal einbauen wollen ("Wer einen Hammer hat...").
- **Hinweis:** Ein "Anti-Pattern" ist *nicht* das Gegenteil eines Design-Patterns.

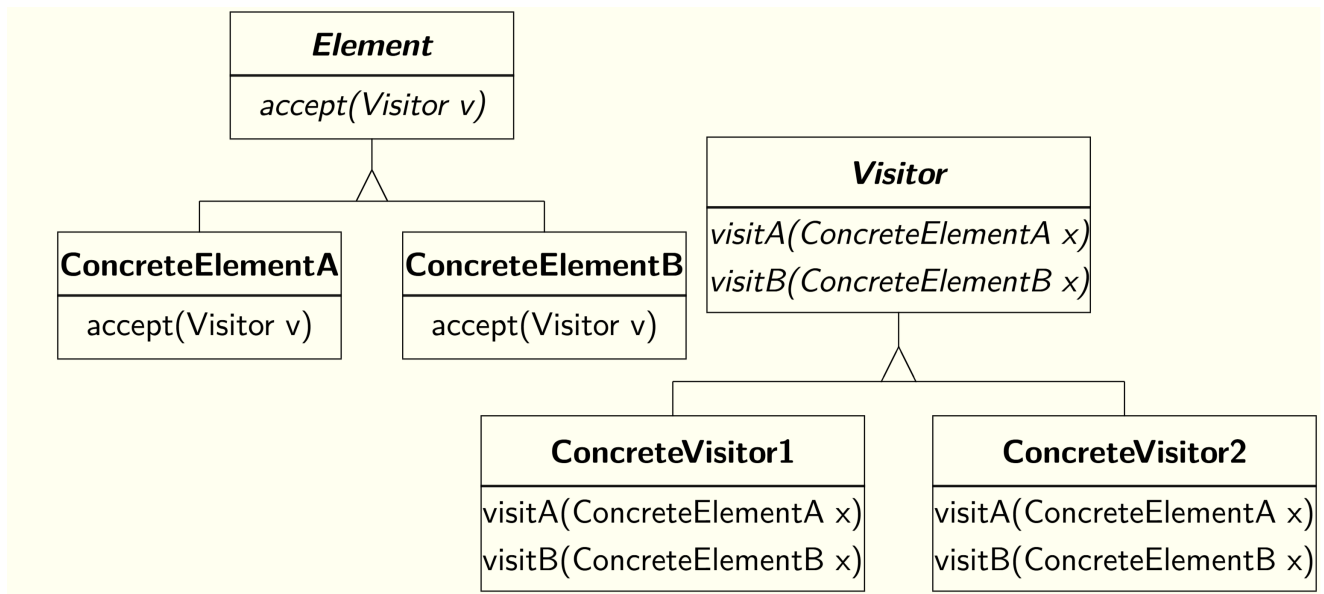
Visitor (Besucher)

Zweck: Ermöglicht mehrfaches dynamisches Binden (Double Dispatch).

Anwendungsgebiete:

- Viele unterschiedliche, nicht verwandte Operationen auf einer Objektstruktur.
- Die Klassen der Objektstruktur ändern sich selten, aber neue Operationen sind häufig nötig.
- Algorithmen arbeiten verteilt über Klassen, sollen aber zentral verwaltet werden.

Struktur:



Eigenschaften:

- **Pro:** Neue Operationen einfach als neue `Visitor`-Unterklassen hinzufügbare. Verwandte Operationen sind zentralisiert.
- **Pro:** Kann mit unabhängigen Klassenhierarchien arbeiten.
- **Contra:** Schlechte Erweiterbarkeit der Element-Klassen (da das Visitor-Interface angepasst werden müsste).
- **Contra:** `visit`-Methoden haben keinen Zugriff auf private Bereiche der Elemente (Kapselungsbruch evtl. nötig).

Implementierung:

- Heute üblich: `accept`-Methoden rufen `visit`-Methoden direkt auf.
- Alternativ: Element-Klassen verwalten dynamische Liste von Visitors (ursprünglicher Ansatz).

Iterator (Cursor)

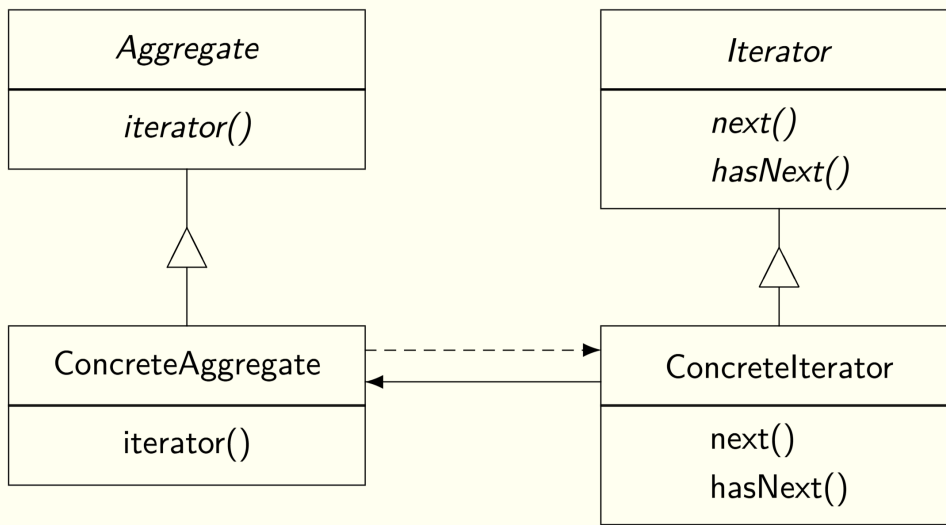
Zweck: Sequentieller Zugriff auf Elemente eines Aggregats, ohne dessen innere Struktur offenzulegen.

Anwendungsgebiete:

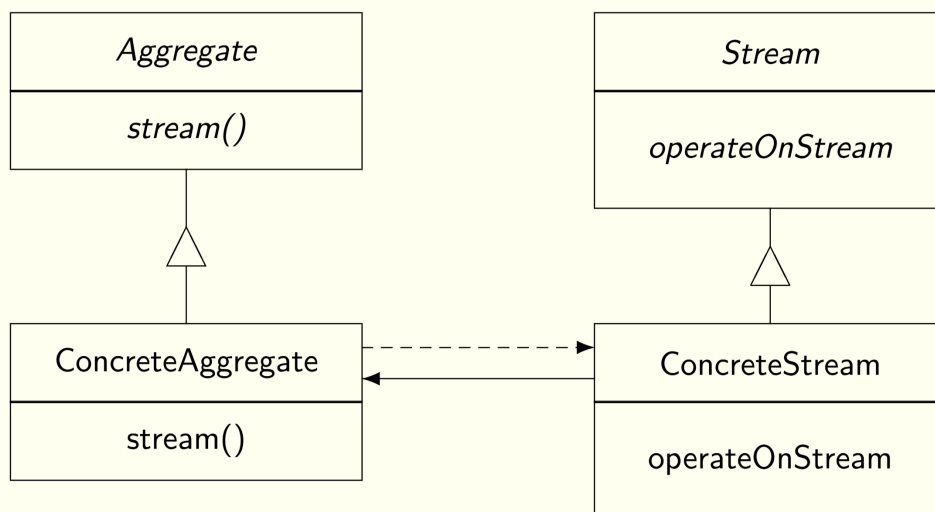
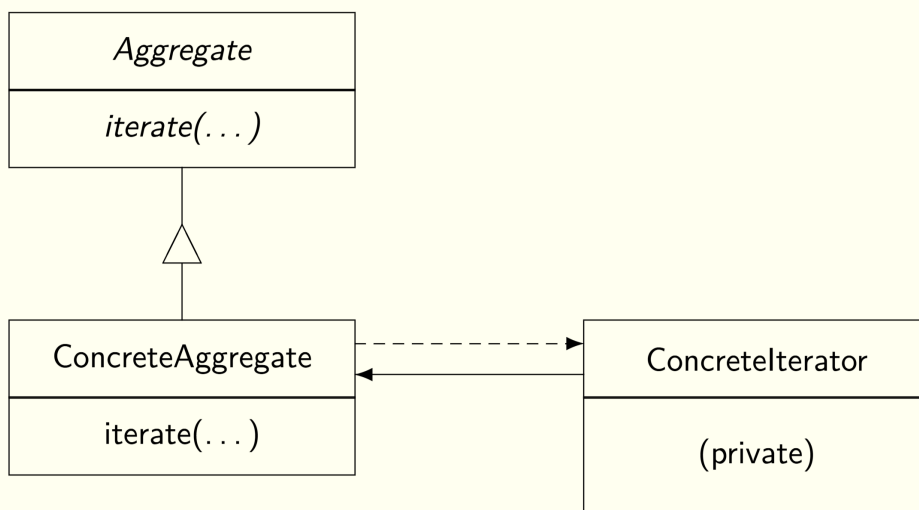
- Kapselung der inneren Darstellung.
- Mehrere gleichzeitige Abarbeitungen des Inhalts möglich.
- Einheitliche Schnittstelle für verschiedene Aggregatstrukturen (Polymorphie).

Strukturen:

Externer Iterator (Client steuert Schleife):



Interner Iterator (Iterator steuert Schleife / Funktional):



Vergleich Extern vs. Intern:

Merkmal	Externer Iterator	Interner Iterator
Steuerung	Schleife außerhalb (Client holt <code>next()</code>)	Schleife im Iterator

Merkmal	Externer Iterator	Interner Iterator
Flexibilität	Flexibler, gut für komplexe Beziehungen zu anderen Objekten	Einfacher in Anwendung, gut für komplexe Beziehungen <i>im</i> Aggregat
Paradigma	Herkömmlich OOP	Funktionale Programmierung
Parallelität	-	Verarbeitung einfach auf Threads aufteilbar

Implementierungshinweise:

- **Robuster Iterator:** Muss Änderungen am Aggregat während der Iteration verkraften.
- Sollte auch auf leeren Aggregaten funktionieren.
- Algorithmus kann im Aggregat oder im Iterator definiert sein.

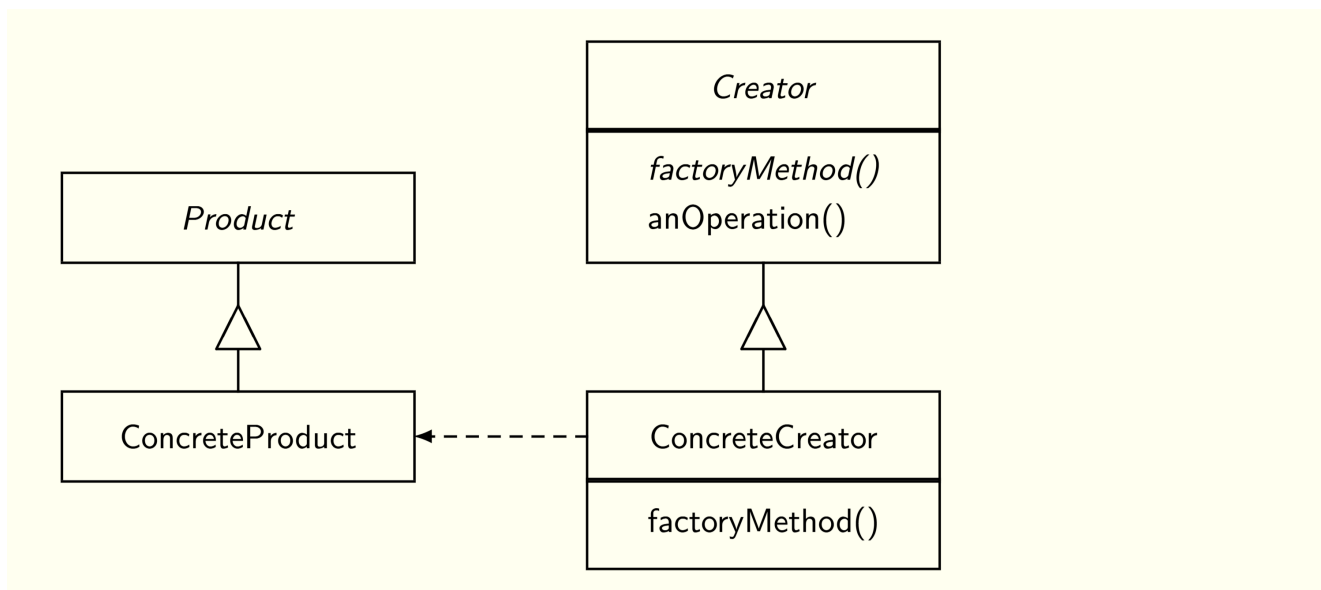
Factory-Method (Virtual-Constructor)

Zweck: Definition einer Schnittstelle zur Objekterzeugung, wobei Unterklassen entscheiden, welche Klasse instanziiert wird.

Anwendungsgebiete:

- Klasse der neuen Objekte ist bei Erzeugung noch unbekannt.
- Delegation der Verantwortlichkeit an Unterklassen (Wissen lokal halten).

Struktur:



Eigenschaften:

- Bietet **Anknüpfungspunkte (Hooks)** für Unterklassen.
- Verknüpft parallele Klassenhierarchien (z.B. `Creator`-Hierarchie parallel zu `Product`-Hierarchie).
 - **Beispiel:** `Tier` (Creator) erzeugt `Futter` (Product). `Rind` erzeugt `Gras`, `Tiger` erzeugt `Fleisch`.

- **Nachteil:** Oft große Anzahl an Unterklassen nötig.

Beispiel (Lazy Initialization):

```
abstract class Creator {
    private Product product = null;
    protected abstract Product createProduct(); // Factory Method

    public Product getProduct() {
        if (product == null)
            product = createProduct(); // Erzeugung erst bei Bedarf
        return product;
    }
}
```

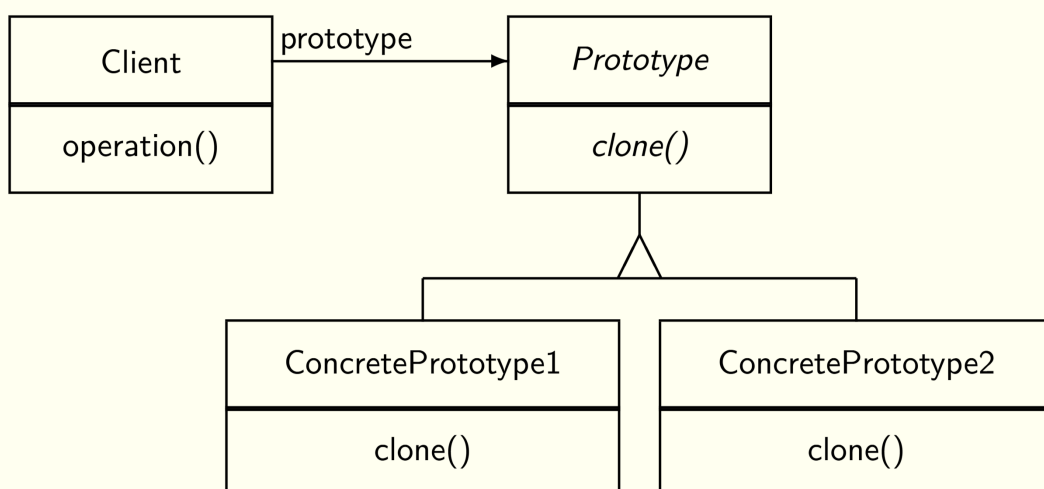
Prototype

Zweck: Objekterzeugung durch Kopieren eines Prototyp-Objekts (statt Instanziierung per `new` und Klasse).

Anwendungsgebiete:

- Klasse des Objekts erst zur Laufzeit bekannt.
- Vermeidung von parallelen Creator-Hierarchien (Unterschied zu Factory-Method).
- Wenn Instanzen nur wenige verschiedene Zustands-Kombinationen haben.
- Kopieren ist oft einfacher als Konstruktoraufruf.

Struktur:



Eigenschaften:

- Versteckt Produkt-Klassen vor dem Anwender.
- **Dynamik:** Prototyp-Menge und die Prototypen selbst sind zur Laufzeit änderbar (im Gegensatz zu statischen Klassenstrukturen).

- Erlaubt dynamische Konfiguration (Verhalten durch Objektkomposition).
- Reduziert die Anzahl nötiger Unterklassen.

Implementierung:

- In Java: `clone()` in `Object` (für flache Kopien, `Cloneable` Interface nötig).
- **Tiefe Kopien:** Schwierig bei zyklischen Strukturen.
- **Initialisierung:** Nach dem Klonen oft `initialize()`-Methoden nötig, da `clone` keine Parameter hat.

Singleton

⚠ Status Wird oft als **Antipattern** betrachtet.

Zweck: Sicherstellen, dass eine Klasse genau eine Instanz hat, und Bereitstellung eines globalen Zugriffspunkts.

Eigenschaften:

- Kontrollierter Zugriff auf die einzige Instanz.
- Vermeidet globale Variablen (im globalen Namensraum).
- Unterstützt Vererbung und ist flexibler als rein statische Methoden (Anzahl der Instanzen könnte später geändert werden) .

Code-Beispiel (Einfach):

```
class Singleton {  
    private static Singleton singleton = null;  
    protected Singleton() {}  
    public static Singleton instance() {  
        if (singleton == null)  
            singleton = new Singleton(); // Lazy Creation  
        return singleton;  
    }  
}
```

Nachteil: Für mehrere alternative Implementierungen (Subclassing) ungeeignet, da `new Singleton()` hartkodiert ist.

Erweiterte Beispiele:

- **Lösung mit Switch/Registry:** Eine statische Variable (`kind`) bestimmt, welche Unterklasse (`SingletonA`, `SingletonB`) instanziiert wird .
- **Lösung mit protected Constructor:** Unterklassen können `instance()` überschreiben oder die Basisklasse nutzt eine Registry .